

Team Notebook

September 27, 2021

Contents

Contents		14 HLD Bhaskar	6	28 Polynomial	13	
1	Advice	2	15 HLD Shubham	7	29 Push-Relabel	17
2	Aho Corasick	2	16 Half Plane Intersection	8	30 Segment Intersection	18
3	Bridge Tree	3	17 Hungarian	9	31 Segment Tree	19
4	CHT	3	18 LIS	9	32 Simplex	20
5	Centroid Decomposition	3	19 Lowest Common Ancestor	9	33 Strongly Connected Component	20
6	Disjoint Set Union	4	20 Manacher	10	34 Suffix Array	21
7	Extended Euclidean GCD	4	21 Min Cost Max Flow	10	35 Template	22
8	FFT	4	22 Minimum Enclosing Circle	11	36 Treap	22
9	Fenwick Tree 2D	4	23 Minkowski	11	37 Two SAT	23
10	Functions	4	24 Mo's algorithm	12	38 Z and KMP	24
11	Gauss Elimination	5	25 NTT	12	39 Z ideas	24
12	General Weighted Matching	5	26 Nearest Pair of Points	13		
13	Geometry	5	27 Persistent Segment Tree	13		

1 Advice

Pre-submit:

Are time limits close? If so, generate max cases. Is the memory usage fine? Could anything overflow? Make sure to submit the right file.

Wrong answer: Print your solution! Print debug output, as well. Are you clearing all datastructures between test cases? Can your algorithm handle the whole range of input?

Read the full problem statement again. Do you handle all corner cases correctly? Have you understood the problem correctly? Any uninitialized variables? Any overflows? Confusing N and M, i and j, etc.? Are you sure your algorithm works? What special cases have you not thought of?

Are you sure the STL functions you use work as you think? Add some assertions, maybe resubmit. Create some testcases to run your algorithm on. Go through the algorithm for a simple case.

Go through this list again. Explain your algorithm to a team mate. Ask the team mate to look at your code. Go for a small walk, e.g. to the toilet. Is your output format correct? Rewrite your solution from the start or let a team mate do it.

Runtime error: Have you tested all corner cases locally? Any uninitialized variables? Are you reading or writing outside the range of any vector? Any assertions that might fail? Any possible division by 0? (mod 0 for example). Any possible infinite recursion? Invalidated pointers or iterators? Are you using too much memory? Debug with resubmits.

Time limit exceeded: Do you have any possible infinite loops? What is the complexity of your algorithm? Are you copying a lot of unnecessary data? (References) How big is the input and output? (consider scanf) Avoid vector, map. (use arrays/unordered_map) What do your team mates think about your algorithm?

Memory limit exceeded: What is the max amount of memory your algorithm should need? Are you clearing all datastructures between test cases?

Primes - 10001st prime is 1299721, 100001st prime is 15485867. Large primes - 999999937, $1e9+7$, 987646789, 987101789; 78498 primes less than 10^6 . The number of divisors of n is at most around 100, for $n \leq 5e4$, 500 for

$n \leq 1e7$, 2000 for $n \leq 1e10$, 200,000 for $n \leq 1e19$. $7! = 5040$, $8! = 40320$, $9! = 362880$, $10! = 3628800$, $11! = 4.0e7$, $12! = 4.8e8$, $15! = 1.3e12$, $20! = 2e18$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

Articulation points and bridges articulation point:- there exist child : $\text{dfslow}[\text{child}] \geq \text{dfsnum}[\text{curr}]$ bridge :- $\text{tree_ed}[\text{dfslow}[\text{ch}]] > \text{dfsnum}[\text{par}]$;

A connected multigraph has an Euler path but not an Euler circuit if and only if it has exactly two vertices of odd degree

Binomial coefficients - base case ncn and $\text{nc0} = 1$; recursion is $\text{nCk} = (\text{n}-1)\text{C}(\text{k}-1) + (\text{n}-1)\text{Ck}$

Catalan numbers - used in valid paranthesis expressions - formula is $\text{Cn} = \sum_{i=0}^{n-1} (\text{CiCn}-i-1)$; Another formula is $\text{Cn} = \frac{2n\text{Cn}}{(n+1)}$. There are Cn binary trees of n nodes and $\text{Cn}-1$ rooted trees of n nodes

Derangements - $\text{D}(n) = (\text{n}-1)(\text{D}(\text{n}-1) + \text{D}(\text{n}-2))$

Burnsides Lemma - number of equivalence classes = $(\sum \text{I}(\text{pi})) / \text{n}$: $\text{I}(\text{pi})$ are number of fixed points. Usual formula: $(\sum_{i=0}^{n-1} \text{k}^{\text{gcd}(i,n)}) / \text{n}$

Stirling numbers - first kind - permutations of n elements with k disjoint cycles. $\text{s}(\text{n}+1, \text{k}) = \text{ns}(\text{n}, \text{k}) + \text{s}(\text{n}, \text{k}-1)$. $\text{s}(0,0) = 1$, $\text{s}(\text{n},0) = 0$ if $\text{n} > 0$. $\sum_{k=0}^n \text{s}(\text{n}, \text{k}) = \text{n}!$

Stirling numbers - Second kind - partition n objects into k non empty subsets. $\text{S}(\text{n}+1, \text{k}) = \text{kS}(\text{n}, \text{k}) + \text{S}(\text{n}, \text{k}-1)$. $\text{S}(0,0) = 1$, $\text{S}(\text{n},0) = 0$ if $\text{n} > 0$. $\text{S}(\text{n}, \text{k}) = (\sum_{j=0}^n (-1)^{(k-j)} \text{kCj} \text{j}^{\text{n}}) / \text{k}!$

Hermite identity - $\sum_{k=0}^{n-1} \text{floor}[(\text{x}+\text{k})/\text{n}] = \text{floor}[\text{nx}]$

Kirchoff matrix tree theorem - number of spanning trees in a graph is determinant of Laplacian Matrix with one row and column removed, where L = degree matrix - adjacency matrix

Expected value tricks:

1. Linearity of Expectation: $\text{E}(\text{X}+\text{Y}) = \text{E}(\text{X}) + \text{E}(\text{Y})$

- Contribution to the sum - If we want to find the sum over many ways/possibilities, we should consider every element (maybe a number, or a pair or an edge) and count how many times it will be added to the answer.
- For independent events - $\text{E}(\text{XY}) = \text{E}(\text{X})\text{E}(\text{Y})$
- Ordered pairs (Super interpretation of square) - The square of the size of a set is equal to the number of ordered pairs of elements in the set. So we iterate over pairs and for each we compute the contribution to the answer. Similarly, the k-th power is equal to the number of sequences (tuples) of length k.
- Powers technique - If you want to maintain the sum of k-th powers, it might help to also maintain the sum of smaller powers. For example, if the sum of 0-th, 1-th and 2-nd powers is S0 , S1 and S2 , and we increase every element by x, the new sums are S0 , $\text{S1} + \text{S0x}$ and $\text{S2} + 2\text{S1x} + \text{x}^2\text{S0}$.

2 Aho Corasick

```
const int N=1e5+5;
string st;
int par[N], nxt[N][26], go[N][26], link[N], elink[N], leaf[N];
char parc[N];
int pt=2;

void add_string(int i){
    int node=1;
    for(auto ch:st){
        int c=ch-'a';
        if(nxt[node][c]==0){
            nxt[node][c]=pt++;
        }
        par[nxt[node][c]]=node;
        parc[nxt[node][c]]=ch;
        node=nxt[node][c];
    }
    leaf[node]=i;
}

int got(int v, char ch);

int get_link(int v){
    if(link[v]==0){
        if(v==1 || par[v]==1){
            link[v]=1;
        }
    }
}
```

```

    }
    else{
        link[v]=got(get_link(par[v]), par[c[v]]);
    }
}
return link[v];
}

int got(int v, char ch){
    int c=ch-'a';
    if(go[v][c]==0){
        if(nxt[v][c]!=0){
            go[v][c]=nxt[v][c];
        }
        else{
            go[v][c] = v==1 ? 1 : got(get_link(v), ch);
        }
    }
    return go[v][c];
}

int exlink(int v){
    if(v==1 || par[v]==1){
        elink[v]=1;
        return elink[v];
    }
    if(elink[v]==0){
        if(leaf[link[v]]){
            elink[v]=link[v];
        }
        else{
            elink[v]=exlink(link[v]);
        }
    }
    return elink[v];
}

```

3 Bridge Tree

```

struct dsu{
    v32 par,rk;
    dsu(){ }
    dsu(int n) {reset(n);}
    void reset(int n){
        rk.assign(n,0);
        par.resize(n);
        iota(all(par),0);}
    int getpar(int i){

```

```

        return (par[i]==i)? i:(par[i]=getpar(par[i]));}
    bool con(int i,int j){
        return getpar(i)==getpar(j);}
    bool join(int i,int j){
        i=getpar(i),j=getpar(j);
        if(i==j) return 0;
        if(rk[i]>rk[j]) par[j]=i;
        else{
            par[i]=j;
            if(rk[i]==rk[j]) rk[j]++;}
        return 1;}};
vector<vector<int> > getBridgeTree(vector<vector<int> > &v,
    int n){
    int in[n]={0},low[n],ctm=0;
    vector<pair<int,int> > bridge;
    dsu d(n);
    function<void(int,int)> dfs=[&](int u,int p){
        in[u]=low[u]=++ctm;
        for(auto &it: v[u]){
            if(it==p) continue;
            if(in[it]){
                if(low[it]<low[u]) low[u]=low[it];
            }else{
                dfs(it,u);
                if(low[it]<low[u]) low[u]=low[it];
                if(low[it]>in[u]) bridge.push_back({u,it});
                else d.join(u,it);}}};
    for(int i=0;i<n;++i) if(!in[i]) dfs(i,i);
    int par[n],id[n];
    for(int i=0;i<n;++i) par[i]=d.getpar(i),id[i]=-1;
    vector<vector<int> > g;
    for(auto &it: bridge){
        it.first=par[it.first],it.second=par[it.second];
        if(id[it.first]==-1){
            id[it.first]=g.size();
            g.push_back(vector<int>(0));}
        if(id[it.second]==-1){
            id[it.second]=g.size();
            g.push_back(vector<int>(0));}
        g[id[it.first]].push_back(id[it.second]);
        g[id[it.second]].push_back(id[it.first]);}
    return g;}

```

4 CHT

```

struct Line {
    mutable int k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }

```

```

    bool operator<(int x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    const int inf = LLONG_MAX;
    int div(int a, int b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end()) { x->p = inf; return false; }
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(int k, int m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    int query(int x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
LineContainer cht;

```

5 Centroid Decomposition

```

vector<set<int> > g;
vector<int> par,sub;
int dfs(int u,int p){
    sub[u]=1;
    for(auto &it: g[u]) if(it!=p) sub[u]+=dfs(it,u);
    return sub[u];}
int find_centroid(int u,int p,int n){
    for(auto &it: g[u]){
        if(it!=p && sub[it]>n/2){
            return find_centroid(it,u,n);}}
    return u;}
void decompose(int u,int p=-1){
    int n=dfs(u,p);
    int centroid=find_centroid(u,p,n);
    if(p!=-1) p=centroid;

```

```
// Do stuff here for merges
// Recurse
par[centroid]=p;
for(auto &it: g[centroid]){
    g[it].erase(centroid);
    decompose(it,centroid);
}
g[centroid].clear();
void reset(int n){
    par.resize(n);
    sub.resize(n);
    g.assign(n,set<int>());}

```

6 Disjoint Set Union

```
int parent[N];
int sz[N];

void make_set(int v) {
    parent[v] = v;
    sz[v] = 1;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (sz[a] < sz[b])
            swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
    }
}

```

7 Extended Euclidean GCD

```
int egcd(int a,int b, int* x, int* y){
    if(a==0){
        *x=0;*y=1;
        return b;}

```

```
int x1,y1;
int gcd=egcd(b%a,a,&x1,&y1);
*x=y1-(b/a)*x1;
*y=x1;
return gcd;}

```

8 FFT

```
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) { //invert = 1 for
    inverse FFT
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }

        if (invert) {
            for (cd & x : a)
                x /= n;
        }
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const
    & b) {
    //function to multiply two polynomials
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end())
        ;
    int n = 1;
    while (n < a.size() + b.size())
        n <= 1;
    fa.resize(n);

```

```
fb.resize(n);
fft(fa, false);
fft(fb, false);
for (int i = 0; i < n; i++)
    fa[i] *= fb[i];
fft(fa, true);
vector<int> result(n);
for (int i = 0; i < n; i++)
    result[i] = round(fa[i].real());
return result;}

```

9 Fenwick Tree 2D

```
struct FenwickTree2D {
    vector<vector<int>> bit;
    int n, m;
    // init(...) { ... }
    int sum(int x, int y) {
        int ret = 0;
        for (int i = x; i >= 0; i = (i & (i + 1)) - 1)
            for (int j = y; j >= 0; j = (j & (j + 1)) - 1)
                ret += bit[i][j];
        return ret;}
    void add(int x, int y, int delta) {
        for (int i = x; i < n; i = i | (i + 1))
            for (int j = y; j < m; j = j | (j + 1))
                bit[i][j] += delta;
    }
};

```

10 Functions

```
#include <bits/stdc++.h>
using namespace std;
#pragma optimize("-O3")
#define int long long int
#define f first
#define s second
#define pb push_back
#define endl '\n'
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

typedef tree<int,null_type,less<int>,rb_tree_tag,
    tree_order_statistics_node_update> indexed_set;
typedef tree<int,null_type,less_equal<int>,rb_tree_tag,
    tree_order_statistics_node_update> indexed_set;

```

```

main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
    for(int i=BS._Find_first();i< BS.size();i = BS._Find_next(i))
        cout<<i<<endl;
    indexed_set se;
    se.find_by_order(2);
    se.order_of_key(7);
    return 0;
}

```

11 Gauss Elimination

```

int gauss (vector <vector<double> > a, vector<double> &ans){
    int n = (int) a.size();
    int m = (int) a[0].size()-1;

    vector<int> where(m,-1);
    for(int col=0, row=0;col<m && row<n; ++col){
        int sel = row;
        for(int i=row;i<n;++i){
            if(abs(a[i][col]) > abs(a[sel][col])){
                sel = i;}}
        if(abs(a[sel][col])<EPS) continue;
        for(int i=col; i<=m; ++i){
            swap(a[sel][i],a[row][i]);}
        where[col] = row;
        for(int i=0;i<n;++i){
            if(i!=row){
                double c = a[i][col]/a[row][col];
                for(int j=col;j<=m;++j){
                    a[i][j] -= a[row][j]*c;}}
            ++row;}
        ans.assign(m,0);
        for(int i=0;i<m;++i){
            if(where[i]!=-1){
                ans[i] = a[where[i]][m]/a[where[i]][i];}}
        for(int i=0;i<n;++i){
            double sum=0;
            for(int j=0;j<m;++j){
                sum+=ans[j]*a[i][j];}
            if(abs(sum-a[i][m])>EPS)
                return 0;}
        for(int i=0;i<m;++i){
            if(where[i]==-1) return MOD;}
    }
}

```

```
return 1;}
```

12 General Weighted Matching

```

struct MaxMatchingEdmonds{
    // Assume General Unweighted Directed Graph
    // O(V^3) edmonds for maximum matching
    vv32 g;
    v32 match,p,base;
    vector<bool> blossom;
    int n;
    int lca(int a,int b){
        vector<bool> used(match.size(),0);
        while(1){
            a=base[a];
            used[a]=1;
            if(match[a]==-1) break;
            a=p[match[a]];}
        while(1){
            b=base[b];
            if(used[b]) return b;
            b=p[match[b]];}}
    void markPath(int v,int b,int children) {
        for(;base[v]!=b;v=p[match[v]]){
            blossom[base[v]]=blossom[base[match[v]]]=1;
            p[v]=children;
            children=match[v];}
    int findPath(int root) {
        vector<bool> used(n,0);
        p.assign(n,-1);
        base.assign(n,0);
        for(int i=0;i<n;++i) base[i] = i;
        used[root]=1;
        int qh=0;
        int qt=0;
        v32 q(n,0);
        q[qt++]=root;
        while(qh<qt){
            int v=q[qh++];
            for(int &to:g[v]){
                if(base[v]==base[to] || match[v]==to) continue;
                if(to==root || match[to]!=-1 && p[match[to]]!=-1)
                    {
                        int curbase=lca(v,to);
                        blossom.assign(n,0);
                        markPath(v,curbase,to);
                        markPath(to,curbase,v);
                        for(int i=0;i<n;++i)

```

```

                        if(blossom[base[i]]){
                            base[i]=curbase;
                            if(!used[i]){
                                used[i]=1;
                                q[qt++]=i;}}
                        }else if(p[to]==-1){
                            p[to]=v;
                            if(match[to]==-1) return to;
                            to=match[to];
                            used[to]=1;
                            q[qt++]=to;}}
                    return -1;}
    int maxMatching(vv32 &graph){
        n=graph.size();
        g=graph;
        match.assign(n,-1);
        p.assign(n,0);
        for(int i=0;i<n;++i){
            if(match[i]==-1){
                int v=findPath(i);
                while(v!=-1){
                    int pv=p[v];
                    int ppv=match[pv];
                    match[v]=pv;
                    match[pv]=v;
                    v=ppv;}}
            int matches=0;
            for(int i=0;i<n;++i) if(match[i]!=-1) ++matches;
            return matches/2;}}

```

13 Geometry

```

#define T long double
#define EPS 1e-9
const int N=1e6;
struct PT{
    T x, y;
    PT() {}
    PT(T x, T y) : x(x), y(y) {}
    PT(const PT& p) : x(p.x), y(p.y) {}
    bool operator < (const PT& rhs) const{
        return make_pair(y, x) < make_pair(rhs.y, rhs.x);
    }
    bool operator == (const PT& rhs) const{
        return make_pair(y, x) == make_pair(rhs.y, rhs.x);
    }
    PT operator + (const PT& p) const{
        return PT(x+p.x, y+p.y);
    }
}

```

```

}
PT operator - (const PT& p) const{
    return PT(x-p.x, y-p.y);
}
PT operator * (T c) const {
    return PT(x*c, y*c);
}
PT operator / (T c) const {
    return PT(x/c, y/c);
}
};

T cross(PT p, PT q){
    return p.x*q.y-p.y*q.x;
}

T area2(PT a, PT b, PT c){
    return cross(a, b)+cross(b, c)+cross(c, a);
}

T dot(PT p, PT q){
    return p.x*q.x+p.y*q.y;
}

T dist2(PT p, PT q){
    return dot(p-q, p-q);
}

bool between(const PT& a, const PT& b, const PT& c){
    return (fabs(area2(a, b, c))<EPS && (a.x-b.x)*(c.x-b.x)<=0
        && (a.y-b.y)*(c.y-b.y)<=0);
}

void Con(vector<PT>& pts){
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    vector<PT> up, dn;
    for(int i=0; i<pts.size(); i++){
        while(up.size()>1 && area2(up[up.size()-2], up.back(), pts[i])>=0){
            up.pop_back();
        }
        while(dn.size()>1 && area2(dn[dn.size()-2], dn.back(), pts[i])<=0){
            dn.pop_back();
        }
        up.push_back(pts[i]);
        dn.push_back(pts[i]);
    }
    pts=dn;

```

```

for(int i=up.size()-2; i>=1; i--){
    pts.push_back(up[i]);
}
if(pts.size()<=2){
    return;
}
dn.clear();
dn.push_back(pts[0]);
dn.push_back(pts[1]);
for(int i=2; i<pts.size(); i++){
    if(between(dn[dn.size()-2], dn[dn.size()-1], pts[i])){
        dn.pop_back();
    }
    dn.push_back(pts[i]);
}
if(dn.size()>=3 && between(dn.back(), dn[0], dn[1])){
    dn[0]=dn.back();
    dn.pop_back();
}
pts=dn;
}

int LP(PT a, PT b, PT c, PT d){
    return fabs(cross(b-a, c-d))<EPS;
}

int LC(PT a, PT b, PT c, PT d){
    return LP(a, b, c, d) && fabs(cross(a-b, a-c)) < EPS &&
        fabs(cross(c-d, c-a))<EPS;
}

int seg(PT a, PT b, PT c, PT d){
    if(LC(a, b, c, d)){
        if(dist2(a, c)<EPS || dist2(a, d)<EPS || dist2(b, c)<EPS
            || dist2(b, d)<EPS){
            return 1;
        }
        if(dot(c-a, c-b)>0 && dot(d-a, d-b)>0 && dot(c-b, d-b)>0){
            return 0;
        }
        return 1;
    }
    if(cross(d-a, b-a)*cross(c-a, b-a)>0){
        return 0;
    }
    if(cross(a-c, d-c)*cross(b-c, d-c)>0){
        return 0;
    }
    return 1;
}

```

```

}

PT lin(PT a, PT b, PT c, PT d){
    b=b-a;
    d=c-d;
    c=c-a;
    return a + b*cross(c, d)/cross(b, d);
}

#define Det(a, b, c) ((b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x))

int PIP(vector<PT>& p, PT q){
    int a=1, b=p.size()-1, c;
    if(Det(p[0], p[a], p[b])>0) swap(a, b);
    if(Det(p[0], p[a], q)>0 || Det(p[0], p[b], q)<0) return 0;
    while(abs(a-b)>1){
        c=(a+b)/2;
        if(Det(p[0], p[c], q)>0) b=c;
        else a=c;
    }
    return Det(p[a], p[b], q)<=0;
}

```

14 HLD Bhaskar

```

struct SegTree{
    v32 T,lazy;
    int N,MX;
    void clear(int n,int mx){
        N=n,MX=mx;
        T.assign(4*N,0);
        lazy.assign(4*N,0);
    }
    void build(int a[],int v,int tl,int tr){
        if(tl==tr){
            T[v]=a[tl];
        }else{
            int tm=(tl+tr)>>1,lf=v<<1,rt=lf^1;
            build(a,lf,tl,tm);
            build(a,rt,tm+1,tr);
            T[v]=min(T[lf],T[rt]);
        }
    }
    void push(int v){
        int lf=v<<1,rt=lf^1;
        T[lf]=(T[lf]+lazy[v]);
        lazy[lf]=(lazy[lf]+lazy[v]);
        T[rt]=(T[rt]+lazy[v]);
        lazy[rt]=(lazy[rt]+lazy[v]);
        lazy[v]=0;
    }
}

```

```

void update(int v,int tl,int tr,int l,int r,int val){
    if(l>r or tl>r or tr<l) return;
    if(l<=tl && tr<=r){
        T[v]=T[v]+val;
        lazy[v]=(lazy[v]+val);}else{
            if(tl==tr) return;
            push(v);
            int tm=(tl+tr)>>1,lf=v<<1,rt=lf^1;;
            update(lf,tl,tm,l,r,val);
            update(rt,tm+1,tr,l,r,val);
            T[v]=max(T[lf],T[rt]);}}
int query(int v,int tl,int tr,int l,int r){
    if(l>r) return MX;
    if(l<=tl && tr<=r) return T[v];
    push(v);
    int tm=(tl+tr)>>1,lf=v<<1,rt=lf^1;
    return max(query(lf,tl,tm,l,min(r,tm)),query(rt,tm+1,tr,
        max(l,tm+1),r));}
int q(int l,int r){
    return query(1,0,N-1,l,r);}
void u(int l,int r,int val){
    update(1,0,N-1,l,r,val);}
} st;
struct hld{
    int n,t;
    v32 sz,in,out,root,par,depth;
    vv32 g;
    SegTree tree;
    void dfs_sz(int v=0,int p=0){
        sz[v]=1;
        for(auto &u: g[v]){
            if(u==p) continue;
            dfs_sz(u,v);
            sz[v]+=sz[u];
            if(sz[u]>sz[g[v][0]]) swap(u, g[v][0]);}}
    void dfs_hld(int v=0,int p=0){
        in[v]=t++;
        par[v]=p;
        depth[v]=depth[p]+1;
        for(auto u: g[v]){
            if(u==p) continue;
            root[u]= (u==g[v][0] ? root[v]:u);
            dfs_hld(u,v);}
        out[v]=t;}
    void pre(vv32 &v){
        g=v;n=v.size();t=0;
        sz.assign(n,0);in.assign(n,0);out.assign(n,0);
        root.assign(n,0);par.assign(n,0);depth.assign(n,0);
        depth[0]=-1;
        dfs_sz();dfs_hld();

```

```

    tree.clear(n,-MOD);}
    template <class BinaryOperation>
    void processPath(int u,int v,BinaryOperation op){
        for(;root[u]!=root[v];v=par[root[v]]){
            if(depth[root[u]] > depth[root[v]]) swap(u,v);
            op(in[root[v]],in[v]); }
            if(depth[u]>depth[v]) swap(u,v);
            op(in[u],in[v]);}
    void modifyPath(int u,int v,const int &value){
        processPath(u,v,[this,&value](int l,int r){tree.u(l,r,
            value);});} // [l,r]
    void modifySubtree(int u,const int &value){
        tree.u(in[u],out[u]-1,value);}
    int queryPath(int u,int v){
        int res=-MOD;
        auto add=[](int &a,const int &b){a=max(a,b);};
        processPath(u,v,[this,&res,&add](int l,int r){add(res,
            tree.q(l,r));});
        return res;}
    int querySubtree(int u){
        return tree.q(in[u],out[u]-1);}
};

```

15 HLD Shubham

```

int n;
v32 adj[LIM];
int p[LIM], in[LIM], out[LIM], sz[LIM], go_up[LIM];
bool vis[LIM];
int t;

void dfs_sz(int node){
    sz[node] = 1;
    forstl(r, adj[node]){
        if(sz[r]>0) continue;
        p[r] = node;
        dfs_sz(r);
        sz[node] += sz[r];
        if(sz[r] > sz[adj[node][0]]){
            swap(r, adj[node][0]);
        }
    }
}

void dfs_hld(int node){
    in[node] = t++;
    vis[node] = 1;
    forstl(r, adj[node]){

```

```

        if(vis[r]) continue;
        go_up[r] = (r == adj[node][0] ? go_up[node] : r);
        dfs_hld(r);
    }
    out[node] = t;
}

struct Segtree{
    v32 t, lazy;

    Segtree(int n) {
        t.assign(4*n, 0);
        lazy.assign(4*n, 0);
    }

    void build(int a[], int v, int tl, int tr){
        if(tl == tr){
            t[v] = a[tl];
        }
        else{
            int tm = (tl + tr)/2;
            build(a, v*2, tl, tm);
            build(a, v*2+1, tm+1, tr);
            t[v] = max(t[v*2], t[v*2+1]);
        }
    }

    void push(int v){
        t[v*2] += lazy[v];
        lazy[v*2] += lazy[v];
        t[v*2+1] += lazy[v];
        lazy[v*2+1] += lazy[v];
        lazy[v] = 0;
    }

    void update(int v, int tl, int tr, int l, int r, int
        addend){
        if(l > r){
            return;
        }
        if(l <= tl && tr <= r){
            t[v] += addend;
            lazy[v] += addend;
        } else {
            push(v);
            int tm = (tl + tr)/2;
            update(v*2, tl, tm, l, min(r, tm), addend);
            update(v*2+1, tm+1, tr, max(l, tm+1), r, addend);
            t[v] = max(t[v*2],t[v*2+1]);
        }
    }
}

```

```

    }

    int query(int v, int tl, int tr, int l, int r) {
        if(l>r) return -MOD;
        if(l <= tl && tr <= r){
            return t[v];
        }
        push(v);
        int tm = (tl + tr)/2;
        return max(query(v*2, tl, tm, l, min(r, tm)), query(v
            *2+1, tm+1, tr, max(l, tm+1), r));
    }
};

Segtree s(LIM);

int max_query(int a, int b){
    int m = -MOD;

    while(go_up[b] != go_up[a]){
        int u = go_up[b], v = b;
        m = max(m, s.query(1, 0, n-1, in[u], in[v]));
        b = p[u];
    }
    m = max(m, s.query(1, 0, n-1, in[a], in[b]));
    return m;
}

void update_subtree(int node, int val){
    // increase value of subtree by val
    int u = in[node], v = out[node] - 1;
    s.update(1, 0, n-1, u, v, val);
}

// in main
dfs_sz(0);
dfs_hld(0);
preprocess();
// now just use hld

```

16 Half Plane Intersection

```

// Redefine epsilon and infinity as necessary. Be mindful of
precision errors.
const long double eps = 1e-9, inf = 1e9;

// Basic point/vector struct.
struct Point {

```

```

    long double x, y;
    explicit Point(long double x = 0, long double y = 0) : x(
        x), y(y) {}

    // Addition, subtraction, multiply by constant, cross
    product.

    friend Point operator + (const Point& p, const Point& q)
    {
        return Point(p.x + q.x, p.y + q.y);
    }

    friend Point operator - (const Point& p, const Point& q)
    {
        return Point(p.x - q.x, p.y - q.y);
    }

    friend Point operator * (const Point& p, const long
        double& k) {
        return Point(p.x * k, p.y * k);
    }

    friend long double cross(const Point& p, const Point& q)
    {
        return p.x * q.y - p.y * q.x;
    }
};

// Basic half-plane struct.
struct Halfplane {

    // 'p' is a passing point of the line and 'pq' is the
    direction vector of the line.
    Point p, pq;
    long double angle;

    Halfplane() {}
    Halfplane(const Point& a, const Point& b) : p(a), pq(b -
        a) {
        angle = atan2l(pq.y, pq.x);
    }

    // Check if point 'r' is outside this half-plane.
    // Every half-plane allows the region to the LEFT of its
    line.
    bool out(const Point& r) {
        return cross(pq, r - p) < -eps;
    }
}

```

```

// Comparator for sorting.
// If the angle of both half-planes is equal, the
leftmost one should go first.
bool operator < (const Halfplane& e) const {
    if (fabsl(angle - e.angle) < eps) return cross(pq, e.
        p - p) < 0;
    return angle < e.angle;
}

// We use equal comparator for std::unique to easily
remove parallel half-planes.
bool operator == (const Halfplane& e) const {
    return fabsl(angle - e.angle) < eps;
}

// Intersection point of the lines of two half-planes. It
is assumed they're never parallel.
friend Point inter(const Halfplane& s, const Halfplane& t
    ) {
    long double alpha = cross((t.p - s.p), t.pq) / cross(
        s.pq, t.pq);
    return s.p + (s.pq * alpha);
}

// Actual algorithm
vector<Point> hp_intersect(vector<Halfplane>& H) {

    Point box[4] = { // Bounding box in CCW order
        Point(inf, inf),
        Point(-inf, inf),
        Point(-inf, -inf),
        Point(inf, -inf)
    };

    for(int i = 0; i<4; i++) { // Add bounding box half-
        planes.
        Halfplane aux(box[i], box[(i+1) % 4]);
        H.push_back(aux);
    }

    // Sort and remove duplicates
    sort(H.begin(), H.end());
    H.erase(unique(H.begin(), H.end()), H.end());

    deque<Halfplane> dq;
    int len = 0;
    for(int i = 0; i < int(H.size()); i++) {

```

```

// Remove from the back of the deque while last half-
// plane is redundant
while (len > 1 && H[i].out(inter(dq[len-1], dq[len
-2]))) {
    dq.pop_back();
    --len;
}

// Remove from the front of the deque while first
// half-plane is redundant
while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
    dq.pop_front();
    --len;
}

// Add new half-plane
dq.push_back(H[i]);
++len;
}

// Final cleanup: Check half-planes at the front against
// the back and vice-versa
while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2]))) {
    dq.pop_back();
    --len;
}

while (len > 2 && dq[len-1].out(inter(dq[0], dq[1]))) {
    dq.pop_front();
    --len;
}

// Report empty intersection if necessary
if (len < 3) return vector<Point>();

// Reconstruct the convex polygon from the remaining half
// -planes.
vector<Point> ret(len);
for(int i = 0; i+1 < len; i++) {
    ret[i] = inter(dq[i], dq[i+1]);
}
ret.back() = inter(dq[len-1], dq[0]);
return ret;
}

```

17 Hungarian

```

struct Hungarian{
    //Important: cost matrix a[1..n][1..m]>=0,n<=m (works with
    // negative costs)
    // 0(V^3) Use p to find matching of 1..m
    vv64 a;
    v64 u,v;
    v32 p,way;
    int n,m;
    Hungarian(int n,int m): n(n),m(m),u(n+1,0),v(m+1,0),p(m
    +1,0),way(m+1,0),a(n+1,v64(m+1,0)){}
    void addEdge(int u,int v,ll val){
        a[u][v]=val;}
    ll solveAssignmentProblem(){
        for(int i=1;i<=n;++i){
            p[0]=i;
            int j0=0;
            v64 minv(m+1,2e17+10);
            vector<bool> used(m+1,0);
            do{
                used[j0] = true;
                int i0=p[j0];
                ll delta=2e17+10;
                int j1=0;
                for(int j=1;j<=m;++j){
                    if(!used[j]){
                        ll cur=a[i0][j]-u[i0]-v[j];
                        if(cur<minv[j]){
                            minv[j]=cur;
                            way[j]=j0;}
                        if(minv[j]<delta){
                            delta=minv[j];
                            j1=j;}}
                for(int j=0;j<=m;++j){
                    if(used[j]){
                        u[p[j]]+=delta;
                        v[j]-=delta;
                    }else minv[j]-=delta;}
                j0=j1;
            }while(p[j0]!=0);
            do{
                int j1=way[j0];
                p[j0]=p[j1];
                j0=j1;
            }while(j0!=0);}
        return -v[0];}};

```

18 LIS

```

int lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;
    for (int i = 0; i < n; i++) {
        int j = upper_bound(d.begin(), d.end(), a[i]) - d.
        begin();
        if (d[j-1] < a[i] && a[i] < d[j])
            d[j] = a[i];}
    int ans = 0;
    for (int i = 0; i <= n; i++) {
        if (d[i] < INF)
            ans = i;}
    return ans;}

```

19 Lowest Common Ancestor

```

int n, l;
vector<vector<int>> adj;

int timer;
vector<int> tin, tout;
vector<vector<int>> up;

void dfs(int v, int p)
{
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i-1]][i-1];

    for (int u : adj[v]) {
        if (u != p)
            dfs(u, v);
    }

    tout[v] = ++timer;
}

bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v)
{

```

```

    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = 1; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}

```

```

void preprocess(int root) {
    tin.resize(n);
    tout.resize(n);
    timer = 0;
    l = ceil(log2(n));
    up.assign(n, vector<int>(l + 1));
    dfs(root, root);
}

```

20 Manacher

```

// Given a string s of length N, finds all palindromes as
// its substrings.
// p[0][i] = half length of longest even palindrome around
// pos i
// p[1][i] = longest odd at i (half rounded down i.e len 2*x
// +1).
//Time: O(N)
void manacher(const string& s){
    int n=s.size();
    v32 p[2]={v32(n+1),v32(n)};
    forn(z,2) for(int i=0,l=0,r=0;i<n;++i){
        int t=r-i+!z;
        if(i<r) p[z][i]=min(t,p[z][l+t]);
        int L=i-p[z][i],R=i+p[z][i]-!z;
        while(L>=1 && R+1<n && s[L]==s[R+1]) p[z][i]++,L--,R++;
        if(R>r) l=L,r=R;}}

```

21 Min Cost Max Flow

```

// Mincost-Maxflow with potentials
// Running Dijkstra after choosing a correct potential
// function

```

```

// Complexity: non-polynomial if the edge capacities are
// large
// If edge capacities are constant, then NlogN if potentials
// are fixed
// If potentials not fixed, read fix_potentials function
// Usage: MCMF m(n), add_edge(a,b,cap,cost), mincost_maxflow
// (s, t)
// In case edge weights are negative, read comments on
// fix_potential below
// need to define int -> ll (long long) since inf = 1e18

```

```

struct edge{
    int v, cap, flow, cost;
};

struct MCMF{
    int n;
    vector<edge> edges;
    vector<vector<int>> adj;
    vector<int> dist, pot, par, flow;
    vector<bool> vis;

```

```

    // Note: you might need to fix potentials beforehand
    // For this, just run fix_potentials(s)
    // If graph is something like line, instead just do pot[i]
    // = -i * 1e12
    // Make sure that initially the edge weights are positive:
    // d(u,v) = pot(u)-pot(v)+w(u,v)

```

```

    int inf = 2e18; // max distance from source to destination

```

```

MCMF(int n){
    this->n = n;
    adj.resize(n);
    dist.assign(n, 0);
    pot.assign(n, 0);
    vis.resize(n);
    par.resize(n);
    flow.resize(n);
}

```

```

void fix_potentials(int s){

```

```

    // Run SPFA to fix the potentials
    // Complexity: O(nm) worst case, O(m) average case
    // In general, run if the edges might have negative costs

```

```

    fill(pot.begin(), pot.end(), inf);
    pot[s] = 0;
    vector<int> cnt(n, 0);

```

```

    vector<bool> in_queue(n, 0);
    queue<int> q;
    q.push(s);
    in_queue[s] = 1;
    while(!q.empty()){
        int v = q.front();
        q.pop();
        in_queue[v] = 0;
        forstl(r, adj[v]){
            if(edges[r].cap == edges[r].flow) continue;
            int to = edges[r].v;
            int len = edges[r].cost;
            if(pot[v] + len < pot[to]){
                pot[to] = pot[v] + len;
                if(!in_queue[to]){
                    q.push(to);
                    in_queue[to] = 1;
                    cnt[to]++;
                    assert(cnt[to] <= n); // negative cycle check
                }
            }
        }
    }
}

```

```

void add_edge(int u, int v, int cap, int cost){
    int sz = edges.size();
    edges.push_back({v, cap, 0, cost});
    edges.push_back({u, cap, cap, -cost});
    adj[u].push_back(sz);
    adj[v].push_back(sz^1);
}

```

```

pair<int,int> mincost_maxflow(int s, int t, int amt){
    int curr_flow = 0, cost = 0;
    while(curr_flow < amt){
        priority_queue<pair<int,int>> q;
        fill(dist.begin(), dist.end(), inf);
        fill(vis.begin(), vis.end(), 0);
        q.push({0, s});
        dist[s] = 0;
        flow[s] = amt - curr_flow;
        while(!q.empty()){
            // running dijkstra to find path
            int curr_dist, v;
            tie(curr_dist, v) = q.top();
            q.pop();
            curr_dist = -curr_dist;
            if(vis[v]) continue;

```

```

vis[v] = 1;
forstl(r, adj[v]){
    int nxt = edges[r].v;
    int new_dist = curr_dist + edges[r].cost + pot[v] - pot[nxt];
    if(vis[nxt]) continue;
    if(edges[r].flow == edges[r].cap) continue;
    if(new_dist >= dist[nxt]) continue;
    dist[nxt] = new_dist;
    par[nxt] = r;
    flow[nxt] = min(flow[v], edges[r].cap - edges[r].flow);
    q.push({-new_dist, nxt});
}
if(!vis[t]) break;
for(int i = 0; i < n; i++){
    dist[i] += pot[i] - pot[s];
}
cost += dist[t] * flow[t];
curr_flow += flow[t];
int last = t;
while(last != s){
    edges[par[last]].flow += flow[t];
    edges[par[last]^1].flow -= flow[t];
    last = edges[par[last]^1].v;
}
dist.swap(pot);
}
return {curr_flow, cost};
}
};

```

22 Minimum Enclosing Circle

```

#include <bits/stdc++.h>
using namespace std;
#pragma optimize("-O3")
#define int long long int
#define f first
#define s second
#define pb push_back
#define endl '\n'
#define double long double

template <class T> int sgn(T x){return (x>0)-(x<0);}
template <class T>
struct Point{
    typedef Point P;

```

```

    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const {return tie(x, y)<tie(p.x, p.y);}
    }
    bool operator==(P p) const {return tie(x, y)==tie(p.x, p.y)}
    ; }
    P operator+(P p) const {return P(x+p.x, y+p.y);}
    P operator-(P p) const {return P(x-p.x, y-p.y);}
    P operator*(T d) const {return P(x*d, y*d);}
    P operator/(T d) const {return P(x/d, y/d);}
    T dot(P p) const {return x*p.x+y*p.y;}
    T cross(P p) const {return x*p.y-y*p.x;}
    T cross(P a, P b) const {return (a-*this).cross(b-*this);}
    T dist2() const {return x*x+y*y;}
    double dist() const {return sqrt((double)dist2());}
    double angle() const {return atan2(y, x);}
    P unit() const {return *this/dist();}
    P perp() const {return P(-y, x);}
    P normal() const {return perp().unit();}
    P rotate(double a) const{
        return P(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a));
    }
    friend ostream& operator<<(ostream& os, P p){
        return os<<"(<p.x<<","<p.y<<")";
    }
};

```

```

typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C){
    return (B-A).dist()*(C-B).dist()*(A-C).dist()/abs((B-A).
        cross(C-A))/2;
}

P ccCenter(const P& A, const P& B, const P& C){
    P b=C-A, c=B-A;
    return A+(b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}

```

```

pair<P, double> mec(vector<P> ps){
    P o=ps[0];
    double r=0, EPS=1+1e-12;
    for(int i=0; i<ps.size(); i++){if((o-ps[i]).dist() > r*EPS){
        o=ps[i], r=0;
        for(int j=0; j<i; j++) if((o-ps[j]).dist() > r*EPS){
            o=(ps[i]+ps[j])/2;
            r=(o-ps[i]).dist();
            for(int k=0; k<j; k++) if((o-ps[k]).dist()>r*EPS){
                o=ccCenter(ps[i], ps[j], ps[k]);
                r=(o-ps[i]).dist();
            }
        }
    }
}

```

```

    }
}
return {o, r};
}

```

```

main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().
        count());
    vector<P>v;
    int t;
    cin>>t;
    int n;
    cin>>n;
    for(int i=0; i<n; i++){
        int a, b;
        cin>>a>>b;
        v.pb(P(b, a*t+b));
    }
    shuffle(v.begin(), v.end(), rng);
    auto u=mec(v);
    cout<<fixed<<setprecision(20)<<(u.s)*(u.s)<<endl;

    return 0;
}

```

23 Minkowski

```

struct pt{
    long long x, y;
    pt operator + (const pt & p) const {
        return pt{x + p.x, y + p.y};
    }
    pt operator - (const pt & p) const {
        return pt{x - p.x, y - p.y};
    }
    long long cross(const pt & p) const {
        return x * p.y - y * p.x;
    }
};

void reorder_polygon(vector<pt> & P){
    size_t pos = 0;
    for(size_t i = 1; i < P.size(); i++){

```

```

        if(P[i].y < P[pos].y || (P[i].y == P[pos].y && P[i].x
        < P[pos].x))
            pos = i;
    }
    rotate(P.begin(), P.begin() + pos, P.end());
}

vector<pt> minkowski(vector<pt> P, vector<pt> Q){
    // the first vertex must be the lowest
    reorder_polygon(P);
    reorder_polygon(Q);
    // we must ensure cyclic indexing
    P.push_back(P[0]);
    P.push_back(P[1]);
    Q.push_back(Q[0]);
    Q.push_back(Q[1]);
    // main part
    vector<pt> result;
    size_t i = 0, j = 0;
    while(i < P.size() - 2 || j < Q.size() - 2){
        result.push_back(P[i] + Q[j]);
        auto cross = (P[i + 1] - P[i]).cross(Q[j + 1] - Q[j])
            ;
        if(cross >= 0)
            ++i;
        if(cross <= 0)
            ++j;
    }
    return result;
}

```

24 Mo's algorithm

```

bool comp(pair<pair<int, int>, int> a, pair<pair<int, int>,
int> b){
    int ga=a.f.f/N;
    int gb=b.f.f/N;
    if(ga==gb){
        return (ga & 1) ? a.f.s <b.f.s : a.f.s > b.f.s;
    }
    else{
        return a.f.f<b.f.f;
    }
}

int cur_l=0;
int cur_r=-1;

```

```

while(cur_l>1){
    cur_l--;
    cnt[arr[cur_l]]++;
}
while(cur_r<r){
    cur_r++;
    cnt[arr[cur_r]]++;
}
while(cur_l<l){
    cnt[arr[cur_l]]--;
    cur_l++;
}
while(cur_r>r){
    cnt[arr[cur_r]]--;
    cur_r--;
}

```

25 NTT

```

const int MOD = 998244353;

int add(int x, int y)
{
    return (x + y) % MOD;
}

int sub(int x, int y)
{
    return add(x, MOD - y);
}

int mul(int x, int y)
{
    return (x * 111 * y) % MOD;
}

int binpow(int x, int y)
{
    int z = 1;
    while(y > 0)
    {
        if(y % 2 == 1)
            z = mul(z, x);
        x = mul(x, x);
        y /= 2;
    }
}

```

```

return z;
}

int inv(int x)
{
    return binpow(x, MOD - 2);
}

int fact[300005];
int ifact[300005];

int C(int n, int k)
{
    return mul(fact[n], (mul(ifact[k], ifact[n - k])));
}

int modpow(int a, int b) {a %= MOD; int res = 1; while (b) {
    if (b & 1)res = mul(res, a); a = mul(a, a); b >>= 1;}
    return res;}
int modinv(int a) {return modpow(a, MOD - 2);}

struct NTT {
    int root, root_1, root_pw = 1, c;
    NTT(int n, bool init = true) {
        while(root_pw < n) root_pw <= 1, N++;
        if (init) {
            vector<int> pf;
            int n = MOD - 1, phi = n;
            for (int i = 2; i * i <= n; i++) {
                if (n % i == 0) {
                    pf.push_back(i);
                    while (n % i == 0) n /= i;
                }
            }
            if (n > 1) pf.push_back(n);
            for (int r = 2; r <= phi; r++) {
                bool ok = true;
                for (auto &x : pf) {
                    if (modpow(r, phi / x) == 1)
                        {ok = false; break;}
                }
                if (ok) {
                    c = (MOD - 1) >> N;
                    root = modpow(r, c), root_1 = modinv(root)
                        ;
                    break;
                }
            }
        }
    }
}

```

```

    }
}
void transform(vector<int> & a, bool invert) {
    int n = a.size();
    assert(n <= root_pw);
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <= 1)
            wlen = (int)(1LL * wlen * wlen % MOD);
        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j], v = (int)(1LL * a[i + j]
                    + len / 2 * w % MOD);
                a[i + j] = u + v < MOD ? u + v : u + v -
                    MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v :
                    u - v + MOD;
                w = (int)(1LL * w * wlen % MOD);
            }
        }
    }
    if (invert) {
        int n_1 = modinv(n);
        for (int & x : a)
            x = (int)(1LL * x * n_1 % MOD);
    }
}
vector<int> multiply(vector<int> const& a, vector<int>
    const& b) {
    vector<int> fa(a), fb(b);
    int n = 1, total = a.size() + b.size();
    while (n < total) n <= 1;
    fa.resize(n);
    fb.resize(n);
    transform(fa, false); transform(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] = (fb[i] * 1LL * fa[i]) % MOD;
    transform(fa, true);
    fa.resize(total - 1);
    while(!fa.empty() && fa.back() == 0) fa.pop_back();
}

```

```

        return fa;
    }
} ntt(800005);

```

26 Nearest Pair of Points

```

vector<pt> t;

void rec(int l, int r) {
    if (r - l <= 3) {
        for (int i = l; i < r; ++i) {
            for (int j = i + 1; j < r; ++j) {
                upd_ans(a[i], a[j]);
            }
            sort(a.begin() + l, a.begin() + r, cmp_y());
            return;
        }

        int m = (l + r) >> 1;
        int midx = a[m].x;
        rec(l, m);
        rec(m, r);
        merge(a.begin() + l, a.begin() + m, a.begin() + m, a.
            begin() + r, t.begin(), cmp_y());
        copy(t.begin(), t.begin() + r - l, a.begin() + l);

        int tsz = 0;
        for (int i = l; i < r; ++i) {
            if (abs(a[i].x - midx) < mindist) {
                for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y <
                    mindist; --j)
                    upd_ans(a[i], t[j]);
                t[tsz++] = a[i];
            }
        }
        // In main, call as:
        t.resize(n);
        sort(a.begin(), a.end(), cmp_x());
        mindist = 1E20;
        rec(0, n);
    }
}

```

27 Persistent Segment Tree

```

int update(int pos, int prev, int tl, int tr){
    int id=++ID;
    if(tl==tr){
        t[id]={1, {1, 1}};
        return id;
    }
}

```

```

int tm=(tl+tr)/2;
l[id]=l[prev];
r[id]=r[prev];
if(pos<=tm){
    l[id]=update(pos, l[id], tl, tm);
}
else{
    r[id]=update(pos, r[id], tm+1, tr);
}
t[id]=merge(t[l[id]], t[r[id]], tl, tm, tr);
return id;
}

```

```

pair<int, pair<int, int>>query(int id, int le, int ri, int
    tl, int tr){
    if(le>ri){
        return {0, {0, 0}};
    }
    if(le==tl && ri==tr){
        return t[id];
    }
    int tm=((tl+tr)/2);
    return merge(query(l[id], le, min(ri, tm), tl, tm), query(r
        [id], max(le, tm+1), ri, tm+1, tr), tl, tm, tr);
}

```

28 Polynomial

```

#include <bits/stdc++.h>

using namespace std;
namespace algebra {
    const int inf = 1e9;
    const int magic = 500; // threshold for sizes to run the
        naive algo

    namespace fft {
        const int maxn = 1 << 18;

        typedef double ftype;
        typedef complex<ftype> point;

        point w[maxn];
        const ftype pi = acos(-1);
        bool initiated = 0;
        void init() {
            if(!initiated) {

```

```

for(int i = 1; i < maxn; i *= 2) {
    for(int j = 0; j < i; j++) {
        w[i + j] = polar(ftype(1), pi * j / i);
    }
}
initiated = 1;
}
}

template<typename T>
void fft(T *in, point *out, int n, int k = 1) {
    if(n == 1) {
        *out = *in;
    } else {
        n /= 2;
        fft(in, out, n, 2 * k);
        fft(in + k, out + n, n, 2 * k);
        for(int i = 0; i < n; i++) {
            auto t = out[i + n] * w[i + n];
            out[i + n] = out[i] - t;
            out[i] += t;
        }
    }
}

template<typename T>
void mul_slow(vector<T> &a, const vector<T> &b) {
    vector<T> res(a.size() + b.size() - 1);
    for(size_t i = 0; i < a.size(); i++) {
        for(size_t j = 0; j < b.size(); j++) {
            res[i + j] += a[i] * b[j];
        }
    }
    a = res;
}

template<typename T>
void mul(vector<T> &a, const vector<T> &b) {
    if(min(a.size(), b.size()) < magic) {
        mul_slow(a, b);
        return;
    }
    init();
    static const int shift = 15, mask = (1 << shift) - 1;
    size_t n = a.size() + b.size() - 1;
    while(__builtin_popcount(n) != 1) {
        n++;
    }
    a.resize(n);
    static point A[maxn], B[maxn];

```

```

    static point C[maxn], D[maxn];
    for(size_t i = 0; i < n; i++) {
        A[i] = point(a[i] & mask, a[i] >> shift);
        if(i < b.size()) {
            B[i] = point(b[i] & mask, b[i] >> shift);
        } else {
            B[i] = 0;
        }
    }
    fft(A, C, n); fft(B, D, n);
    for(size_t i = 0; i < n; i++) {
        point c0 = C[i] + conj(C[(n - i) % n]);
        point c1 = C[i] - conj(C[(n - i) % n]);
        point d0 = D[i] + conj(D[(n - i) % n]);
        point d1 = D[i] - conj(D[(n - i) % n]);
        A[i] = c0 * d0 - point(0, 1) * c1 * d1;
        B[i] = c0 * d1 + d0 * c1;
    }
    fft(A, C, n); fft(B, D, n);
    reverse(C + 1, C + n);
    reverse(D + 1, D + n);
    int t = 4 * n;
    for(size_t i = 0; i < n; i++) {
        int64_t A0 = llround(real(C[i]) / t);
        T A1 = llround(imag(D[i]) / t);
        T A2 = llround(imag(C[i]) / t);
        a[i] = A0 + (A1 << shift) + (A2 << 2 * shift);
    }
    return;
}

template<typename T>
T bpow(T x, size_t n) {
    return n ? n % 2 ? x * bpow(x, n - 1) : bpow(x * x, n / 2) : T(1);
}

template<typename T>
T bpow(T x, size_t n, T m) {
    return n ? n % 2 ? x * bpow(x, n - 1, m) % m : bpow(x * x % m, n / 2, m) : T(1);
}

template<typename T>
T gcd(const T &a, const T &b) {
    return b == T(0) ? a : gcd(b, a % b);
}

template<typename T>
T nCr(T n, int r) { // runs in O(r)
    T res(1);
    for(int i = 0; i < r; i++) {
        res *= (n - T(i));
    }

```

```

    res /= (i + 1);
}
}
return res;
}

template<int m>
struct modular {
    int64_t r;
    modular() : r(0) {}
    modular(int64_t rr) : r(rr) {if(abs(r) >= m) r %= m; if(r < 0) r += m;}
    modular inv() const {return bpow(*this, m - 2);}
    modular operator * (const modular &t) const {return (r * t.r) % m;}
    modular operator / (const modular &t) const {return *this * t.inv();}
    modular operator += (const modular &t) {r += t.r; if(r >= m) r -= m; return *this;}
    modular operator -= (const modular &t) {r -= t.r; if(r < 0) r += m; return *this;}
    modular operator + (const modular &t) const {return modular(*this) += t;}
    modular operator - (const modular &t) const {return modular(*this) -= t;}
    modular operator *= (const modular &t) {return *this = *this * t;}
    modular operator /= (const modular &t) {return *this = *this / t;}

    bool operator == (const modular &t) const {return r == t.r;}
    bool operator != (const modular &t) const {return r != t.r;}

    operator int64_t() const {return r;}
};

template<int T>
istream& operator >> (istream &in, modular<T> &x) {
    return in >> x.r;
}

template<typename T>
struct poly {
    vector<T> a;

    void normalize() { // get rid of leading zeroes
        while(!a.empty() && a.back() == T(0)) {
            a.pop_back();
        }
    }

```

```

}

poly(){}
poly(T a0) : a{a0}{normalize();}
poly(vector<T> t) : a(t){normalize();}

poly operator += (const poly &t) {
    a.resize(max(a.size(), t.a.size()));
    for(size_t i = 0; i < t.a.size(); i++) {
        a[i] += t.a[i];
    }
    normalize();
    return *this;
}

poly operator -= (const poly &t) {
    a.resize(max(a.size(), t.a.size()));
    for(size_t i = 0; i < t.a.size(); i++) {
        a[i] -= t.a[i];
    }
    normalize();
    return *this;
}

poly operator + (const poly &t) const {return poly(*this)
    += t;}
poly operator - (const poly &t) const {return poly(*this)
    -= t;}

poly mod_xk(size_t k) const { // get same polynomial mod x
    ^k
    k = min(k, a.size());
    return vector<T>(begin(a), begin(a) + k);
}

poly mul_xk(size_t k) const { // multiply by x^k
    poly res(*this);
    res.a.insert(begin(res.a), k, 0);
    return res;
}

poly div_xk(size_t k) const { // divide by x^k, dropping
    coefficients
    k = min(k, a.size());
    return vector<T>(begin(a) + k, end(a));
}

poly substr(size_t l, size_t r) const { // return mod_xk(r)
    ).div_xk(l)
    l = min(l, a.size());
    r = min(r, a.size());
    return vector<T>(begin(a) + l, begin(a) + r);
}

poly inv(size_t n) const { // get inverse series mod x^n
    assert(!is_zero());

```

```

    poly ans = a[0].inv();
    size_t a = 1;
    while(a < n) {
        poly C = (ans * mod_xk(2 * a)).substr(a, 2 * a);
        ans -= (ans * C).mod_xk(a).mul_xk(a);
        a *= 2;
    }
    return ans.mod_xk(n);
}

poly operator *= (const poly &t) {fft::mul(a, t.a);
    normalize(); return *this;}
poly operator * (const poly &t) const {return poly(*this)
    *= t;}

poly reverse(size_t n, bool rev = 0) const { // reverses
    and leaves only n terms
    poly res(*this);
    if(rev) { // If rev = 1 then tail goes to head
        res.a.resize(max(n, res.a.size()));
    }
    std::reverse(res.a.begin(), res.a.end());
    return res.mod_xk(n);
}

pair<poly, poly> divmod_slow(const poly &b) const { //
    when divisor or quotient is small
    vector<T> A(a);
    vector<T> res;
    while(A.size() >= b.a.size()) {
        res.push_back(A.back() / b.a.back());
        if(res.back() != T(0)) {
            for(size_t i = 0; i < b.a.size(); i++) {
                A[A.size() - i - 1] -= res.back() * b.a[b.a.size() - i
                    - 1];
            }
        }
        A.pop_back();
    }
    std::reverse(begin(res), end(res));
    return {res, A};
}

pair<poly, poly> divmod(const poly &b) const { // returns
    quotient and remainder of a mod b
    if(deg() < b.deg()) {
        return {poly{0}, *this};
    }
    int d = deg() - b.deg();
    if(min(d, b.deg()) < magic) {

```

```

        return divmod_slow(b);
    }
    poly D = (reverse(d + 1) * b.reverse(d + 1).inv(d + 1)).
        mod_xk(d + 1).reverse(d + 1, 1);
    return {D, *this - D * b};
}

poly operator / (const poly &t) const {return divmod(t).
    first;}
poly operator % (const poly &t) const {return divmod(t).
    second;}
poly operator /= (const poly &t) {return *this = divmod(t)
    .first;}
poly operator %= (const poly &t) {return *this = divmod(t)
    .second;}
poly operator *= (const T &x) {
    for(auto &it: a) {
        it *= x;
    }
    normalize();
    return *this;
}
poly operator /= (const T &x) {
    for(auto &it: a) {
        it /= x;
    }
    normalize();
    return *this;
}
poly operator * (const T &x) const {return poly(*this) *=
    x;}
poly operator / (const T &x) const {return poly(*this) /=
    x;}

void print() const {
    for(auto it: a) {
        cout << it << ' ';
    }
    cout << endl;
}

T eval(T x) const { // evaluates in single point x
    T res(0);
    for(int i = int(a.size()) - 1; i >= 0; i--) {
        res *= x;
        res += a[i];
    }
    return res;
}

T& lead() { // leading coefficient

```

```

    return a.back();
}
int deg() const { // degree
    return a.empty() ? -inf : a.size() - 1;
}
bool is_zero() const { // is polynomial zero
    return a.empty();
}
T operator [](int idx) const {
    return idx >= (int)a.size() || idx < 0 ? T(0) : a[idx];
}

T& coef(size_t idx) { // mutable reference at coefficient
    return a[idx];
}
bool operator == (const poly &t) const {return a == t.a;}
bool operator != (const poly &t) const {return a != t.a;}

poly deriv() { // calculate derivative
    vector<T> res;
    for(int i = 1; i <= deg(); i++) {
        res.push_back(T(i) * a[i]);
    }
    return res;
}
poly integr() { // calculate integral with C = 0
    vector<T> res = {0};
    for(int i = 0; i <= deg(); i++) {
        res.push_back(a[i] / T(i + 1));
    }
    return res;
}
size_t leading_xk() const { // Let  $p(x) = x^k * t(x)$ ,
    return k
    if(is_zero()) {
        return inf;
    }
    int res = 0;
    while(a[res] == T(0)) {
        res++;
    }
    return res;
}
poly log(size_t n) { // calculate  $\log p(x) \bmod x^n$ 
    assert(a[0] == T(1));
    return (deriv().mod_xk(n) * inv(n)).integr().mod_xk(n);
}
poly exp(size_t n) { // calculate  $\exp p(x) \bmod x^n$ 
    if(is_zero()) {
        return T(1);

```

```

    }
    assert(a[0] == T(0));
    poly ans = T(1);
    size_t a = 1;
    while(a < n) {
        poly C = ans.log(2 * a).div_xk(a) - substr(a, 2 * a);
        ans -= (ans * C).mod_xk(a).mul_xk(a);
        a *= 2;
    }
    return ans.mod_xk(n);
}
poly pow_slow(size_t k, size_t n) { // if k is small
    return k % 2 ? (*this * pow_slow(k - 1, n)).mod_xk(n)
        : (*this * *this).mod_xk(n).pow_slow(k / 2, n) : T(1);
}
poly pow(size_t k, size_t n) { // calculate  $p^k(x) \bmod x^n$ 
    if(is_zero()) {
        return *this;
    }
    if(k < magic) {
        return pow_slow(k, n);
    }
    int i = leading_xk();
    T j = a[i];
    poly t = div_xk(i) / j;
    return bpow(j, k) * (t.log(n) * T(k)).exp(n).mul_xk(i * k)
        .mod_xk(n);
}
poly mulx(T x) { // component-wise multiplication with  $x^k$ 
    T cur = 1;
    poly res(*this);
    for(int i = 0; i <= deg(); i++) {
        res.coef(i) *= cur;
        cur *= x;
    }
    return res;
}
poly mulx_sq(T x) { // component-wise multiplication with  $x^{k^2}$ 
    T cur = x;
    T total = 1;
    T xx = x * x;
    poly res(*this);
    for(int i = 0; i <= deg(); i++) {
        res.coef(i) *= total;
        total *= cur;
        cur *= xx;
    }

```

```

    return res;
}
vector<T> chirpz_even(T z, int n) { //  $P(1), P(z^2), P(z^4), \dots, P(z^{2(n-1)})$ 
    int m = deg();
    if(is_zero()) {
        return vector<T>(n, 0);
    }
    vector<T> vv(m + n);
    T zi = z.inv();
    T zz = zi * zi;
    T cur = zi;
    T total = 1;
    for(int i = 0; i <= max(n - 1, m); i++) {
        if(i <= m) {vv[m - i] = total;}
        if(i < n) {vv[m + i] = total;}
        total *= cur;
        cur *= zz;
    }
    poly w = (mulx_sq(z) * vv).substr(m, m + n).mulx_sq(z);
    vector<T> res(n);
    for(int i = 0; i < n; i++) {
        res[i] = w[i];
    }
    return res;
}
vector<T> chirpz(T z, int n) { //  $P(1), P(z), P(z^2), \dots, P(z^{n-1})$ 
    auto even = chirpz_even(z, (n + 1) / 2);
    auto odd = mulx(z).chirpz_even(z, n / 2);
    vector<T> ans(n);
    for(int i = 0; i < n / 2; i++) {
        ans[2 * i] = even[i];
        ans[2 * i + 1] = odd[i];
    }
    if(n % 2 == 1) {
        ans[n - 1] = even.back();
    }
    return ans;
}
template<typename iter>
vector<T> eval(vector<poly> &tree, int v, iter l, iter r)
    { // auxiliary evaluation function
    if(r - l == 1) {
        return {eval(*l)};
    } else {
        auto m = l + (r - l) / 2;
        auto A = (*this % tree[2 * v]).eval(tree, 2 * v, l, m);
        auto B = (*this % tree[2 * v + 1]).eval(tree, 2 * v + 1, m, r);

```

```

    A.insert(end(A), begin(B), end(B));
    return A;
}
}
vector<T> eval(vector<T> x) { // evaluate polynomial in (
    x1, ..., xn)
    int n = x.size();
    if(is_zero()) {
        return vector<T>(n, T(0));
    }
    vector<poly> tree(4 * n);
    build(tree, 1, begin(x), end(x));
    return eval(tree, 1, begin(x), end(x));
}
template<typename iter>
poly inter(vector<poly> &tree, int v, iter l, iter r, iter
    ly, iter ry) { // auxiliary interpolation function
    if(r - l == 1) {
        return {*ly / a[0]};
    } else {
        auto m = l + (r - l) / 2;
        auto my = ly + (ry - ly) / 2;
        auto A = (*this % tree[2 * v]).inter(tree, 2 * v, l, m,
            ly, my);
        auto B = (*this % tree[2 * v + 1]).inter(tree, 2 * v + 1,
            m, r, my, ry);
        return A * tree[2 * v + 1] + B * tree[2 * v];
    }
}
};
template<typename T>
poly<T> operator * (const T& a, const poly<T>& b) {
    return b * a;
}

template<typename T>
poly<T> xk(int k) { // return x^k
    return poly<T>{1}.mul_xk(k);
}

template<typename T>
T resultant(poly<T> a, poly<T> b) { // computes resultant
    of a and b
    if(b.is_zero()) {
        return 0;
    } else if(b.deg() == 0) {
        return bpow(b.lead(), a.deg());
    } else {
        int pw = a.deg();
        a %= b;

```

```

        pw -= a.deg();
        T mul = bpow(b.lead(), pw) * T((b.deg() & a.deg() & 1) ?
            -1 : 1);
        T ans = resultant(b, a);
        return ans * mul;
    }
}
template<typename iter>
poly<typename iter::value_type> kmul(iter L, iter R) { //
    computes (x-a1)(x-a2)...(x-an) without building tree
    if(R - L == 1) {
        return vector<typename iter::value_type>{-*L, 1};
    } else {
        iter M = L + (R - L) / 2;
        return kmul(L, M) * kmul(M, R);
    }
}
template<typename T, typename iter>
poly<T> build(vector<poly<T>> &res, int v, iter L, iter R)
    { // builds evaluation tree for (x-a1)(x-a2)...(x-an)
    if(R - L == 1) {
        return res[v] = vector<T>{-*L, 1};
    } else {
        iter M = L + (R - L) / 2;
        return res[v] = build(res, 2 * v, L, M) * build(res, 2 *
            v + 1, M, R);
    }
}
template<typename T>
poly<T> inter(vector<T> x, vector<T> y) { // interpolates
    minimum polynomial from (xi, yi) pairs
    int n = x.size();
    vector<poly<T>> tree(4 * n);
    return build(tree, 1, begin(x), end(x)).deriv().inter(tree
        , 1, begin(x), end(x), begin(y), end(y));
}
};
using namespace algebra;

const int mod = 1e9 + 7;
typedef modular<mod> base;
typedef poly<base> polyn;

using namespace algebra;

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n = 100000;

```

```

    polyn a;
    vector<base> x;
    for(int i = 0; i <= n; i++) {
        a.a.push_back(1 + rand() % 100);
        x.push_back(1 + rand() % (2 * n));
    }
    sort(begin(x), end(x));
    x.erase(unique(begin(x), end(x)), end(x));
    auto b = a.eval(x);
    cout << clock() / double(CLOCKS_PER_SEC) << endl;
    auto c = inter(x, b);
    polyn md = kmul(begin(x), end(x));
    cout << clock() / double(CLOCKS_PER_SEC) << endl;
    assert(c == a % md);
    return 0;
}

```

29 Push-Relabel

```

//Push-Relabel Algorithm for Flows - Gap Heuristic,
    Complexity: O(V^3)
//To obtain the actual flow values, look at all edges with
    capacity > 0
//Zero capacity edges are residual edges
struct edge{
    int from, to, cap, flow, index;
    edge(int from, int to, int cap, int flow, int index):
        from(from), to(to), cap(cap), flow(flow), index(index)
        {};
}
struct PushRelabel{
    int n;
    vector<vector<edge>> > g;
    vector<long long> excess;
    vector<int> height, active, count;
    queue<int> Q;
    PushRelabel(int n): n(n), g(n), excess(n), height(n), active(n)
        , count(2*n) {}
    void addEdge(int from, int to, int cap){
        g[from].push_back(edge(from, to, cap, 0, g[to].size()));
        if(from==to) g[from].back().index++;
        g[to].push_back(edge(to, from, 0, 0, g[from].size()-1));
    }
    void enqueue(int v){
        if(!active[v] && excess[v]>0){
            active[v]=true;
            Q.push(v);
        }
    }
    void push(edge &e){
        int amt=(int)min(excess[e.from], (long long)e.cap - e.flow)
        ;
    }
}

```

```

if(height[e.from]<=height[e.to] || amt==0) return;
e.flow += amt;
g[e.to][e.index].flow -= amt;
excess[e.to] += amt;
excess[e.from] -= amt;
enqueue(e.to);}
void relabel(int v){
    count[height[v]]--;
    int d=2*n;
    for(auto &it:g[v]){
        if(it.cap-it.flow>0) d=min(d, height[it.to]+1);}
    height[v]=d;
    count[height[v]]++;
    enqueue(v);}
void gap(int k){
    for(int v=0;v<n;v++){
        if(height[v]<k) continue;
        count[height[v]]--;
        height[v]=max(height[v], n+1);
        count[height[v]]++;
        enqueue(v);}}
void discharge(int v){
    for(int i=0; excess[v]>0 && i<g[v].size(); i++) push(g[v][i]);
    if(excess[v]>0){
        if(count[height[v]]==1) gap(height[v]);
        else relabel(v);}}
long long max_flow(int source, int dest){
    count[0] = n-1;
    count[n] = 1;
    height[source] = n;
    active[source] = active[dest] = 1;
    for(auto &it:g[source]){
        excess[source]+=it.cap;
        push(it);}
    while(!Q.empty()){
        int v=Q.front();
        Q.pop();
        active[v]=false;
        discharge(v);}
    long long max_flow=0;
    for(auto &e:g[source]) max_flow+=e.flow;
    return max_flow;};

```

```

struct pt {
    double x, y;
};

struct seg {
    pt p, q;
    int id;

    double get_y(double x) const {
        if (abs(p.x - q.x) < EPS)
            return p.y;
        return p.y + (q.y - p.y) * (x - p.x) / (q.x - p.x);
    }
};

bool intersect1d(double l1, double r1, double l2, double r2)
{
    if (l1 > r1)
        swap(l1, r1);
    if (l2 > r2)
        swap(l2, r2);
    return max(l1, l2) <= min(r1, r2) + EPS;
}

int vec(const pt& a, const pt& b, const pt& c) {
    double s = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    return abs(s) < EPS ? 0 : s > 0 ? +1 : -1;
}

bool intersect(const seg& a, const seg& b)
{
    return intersect1d(a.p.x, a.q.x, b.p.x, b.q.x) &&
        intersect1d(a.p.y, a.q.y, b.p.y, b.q.y) &&
        vec(a.p, a.q, b.p) * vec(a.p, a.q, b.q) <= 0 &&
        vec(b.p, b.q, a.p) * vec(b.p, b.q, a.q) <= 0;
}

bool operator<(const seg& a, const seg& b)
{
    double x = max(min(a.p.x, a.q.x), min(b.p.x, b.q.x));
    return a.get_y(x) < b.get_y(x) - EPS;
}

struct event {
    double x;
    int tp, id;

    event() {}
    event(double x, int tp, int id) : x(x), tp(tp), id(id) {}

```

```

    bool operator<(const event& e) const {
        if (abs(x - e.x) > EPS)
            return x < e.x;
        return tp > e.tp;
    }
};

set<seg> s;
vector<set<seg>::iterator> where;

set<seg>::iterator prev(set<seg>::iterator it) {
    return it == s.begin() ? s.end() : --it;
}

set<seg>::iterator next(set<seg>::iterator it) {
    return ++it;
}

pair<int, int> solve(const vector<seg>& a) {
    int n = (int)a.size();
    vector<event> e;
    for (int i = 0; i < n; ++i) {
        e.push_back(event(min(a[i].p.x, a[i].q.x), +1, i));
        e.push_back(event(max(a[i].p.x, a[i].q.x), -1, i));
    }
    sort(e.begin(), e.end());

    s.clear();
    where.resize(a.size());
    for (size_t i = 0; i < e.size(); ++i) {
        int id = e[i].id;
        if (e[i].tp == +1) {
            set<seg>::iterator nxt = s.lower_bound(a[id]),
                prv = prev(nxt);
            if (nxt != s.end() && intersect(*nxt, a[id]))
                return make_pair(nxt->id, id);
            if (prv != s.end() && intersect(*prv, a[id]))
                return make_pair(prv->id, id);
            where[id] = s.insert(nxt, a[id]);
        } else {
            set<seg>::iterator nxt = next(where[id]), prv =
                prev(where[id]);
            if (nxt != s.end() && prv != s.end() && intersect
                (*nxt, *prv))
                return make_pair(prv->id, nxt->id);
            s.erase(where[id]);
        }
    }
}

```

30 Segment Intersection

```
const double EPS = 1E-9;
```

```
    return make_pair(-1, -1);
}
```

31 Segment Tree

// Normal Segment Tree

```
void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = t[v*2] + t[v*2+1];
    }
}

int sum(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && r == tr) {
        return t[v];
    }
    int tm = (tl + tr) / 2;
    return sum(v*2, tl, tm, l, min(r, tm))
        + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
}

void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        t[v] = new_val;
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = t[v*2] + t[v*2+1];
    }
}

//Merge Sort Tree

vector<int> t[4*MAXN];

void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
```

```
        t[v] = vector<int>(1, a[tl]);
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        merge(t[v*2].begin(), t[v*2].end(), t[v*2+1].begin(),
            t[v*2+1].end(),
            back_inserter(t[v]));
    }
}

int query(int v, int tl, int tr, int l, int r, int x) {
    if (l > r)
        return INF;
    if (l == tl && r == tr) {
        vector<int>::iterator pos = lower_bound(t[v].begin(),
            t[v].end(), x);
        if (pos != t[v].end())
            return *pos;
        return INF;
    }
    int tm = (tl + tr) / 2;
    return min(query(v*2, tl, tm, l, min(r, tm), x),
        query(v*2+1, tm+1, tr, max(l, tm+1), r, x));
}

void update(int v, int tl, int tr, int pos, int new_val) {
    t[v].erase(t[v].find(a[pos]));
    t[v].insert(new_val);
    if (tl != tr) {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
    } else {
        a[pos] = new_val;
    }
}

//lazy value update, query value at a point

void push(int v) {
    if (marked[v]) {
        t[v*2] = t[v*2+1] = t[v];
        marked[v*2] = marked[v*2+1] = true;
        marked[v] = false;
    }
}
```

```
void update(int v, int tl, int tr, int l, int r, int new_val)
{
    if (l > r)
        return;
    if (l == tl && tr == r) {
        t[v] = new_val;
        marked[v] = true;
    } else {
        push(v);
        int tm = (tl + tr) / 2;
        update(v*2, tl, tm, l, min(r, tm), new_val);
        update(v*2+1, tm+1, tr, max(l, tm+1), r, new_val);
    }
}

int get(int v, int tl, int tr, int pos) {
    if (tl == tr) {
        return t[v];
    }
    push(v);
    int tm = (tl + tr) / 2;
    if (pos <= tm)
        return get(v*2, tl, tm, pos);
    else
        return get(v*2+1, tm+1, tr, pos);
}

//lazy addition, query for maximum

void push(int v) {
    t[v*2] += lazy[v];
    lazy[v*2] += lazy[v];
    t[v*2+1] += lazy[v];
    lazy[v*2+1] += lazy[v];
    lazy[v] = 0;
}

void update(int v, int tl, int tr, int l, int r, int addend)
{
    if (l > r)
        return;
    if (l == tl && tr == r) {
        t[v] += addend;
        lazy[v] += addend;
    } else {
        push(v);
        int tm = (tl + tr) / 2;
        update(v*2, tl, tm, l, min(r, tm), addend);
        update(v*2+1, tm+1, tr, max(l, tm+1), r, addend);
        t[v] = max(t[v*2], t[v*2+1]);
    }
}
```

```

    }
}

int query(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return -INF;
    if (l <= tl && tr <= r)
        return t[v];
    push(v);
    int tm = (tl + tr) / 2;
    return max(query(v*2, tl, tm, l, min(r, tm)),
               query(v*2+1, tm+1, tr, max(l, tm+1), r));
}

```

32 Simplex

```

// Two-phase simplex algorithm for solving linear programs
// of the form
//      maximize   c^T x
//      subject to  Ax <= b
//                  x >= 0
// INPUT: A -- an m x n matrix
//         b -- an m-dimensional vector
//         c -- an n-dimensional vector
//         x -- a vector where the optimal solution will be
//              stored
// OUTPUT: value of the optimal solution (infinity if
//         unbounded
//         above, nan if infeasible)
// To use this code, create an LPSolver object with A, b,
// and c as
// arguments. Then, call Solve(x).
typedef long double DOUBLE;
typedef vector<DOUBLE> VD;
typedef vector<VD> VVD;
typedef vector<int> VI;
const DOUBLE EPS = 1e-9;

struct LPSolver {
    int m, n;
    VI B, N;
    VVD D;

    LPSolver(const VVD &A, const VD &b, const VD &c) :
        m(b.size()), n(c.size()), N(n + 1), B(m), D(m + 2, VD(n + 2)) {
        for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) D
            [i][j] = A[i][j];

```

```

        for (int i = 0; i < m; i++) { B[i] = n + i; D[i][n] = -1;
            D[i][n + 1] = b[i]; }
        for (int j = 0; j < n; j++) { N[j] = j; D[m][j] = -c[j];
            }
        N[n] = -1; D[m + 1][n] = 1;

    void Pivot(int r, int s) {
        for (int i = 0; i < m + 2; i++) if (i != r)
            for (int j = 0; j < n + 2; j++) if (j != s)
                D[i][j] -= D[r][j] * D[i][s] / D[r][s];
        for (int j = 0; j < n + 2; j++) if (j != s) D[r][j] /= D[r][s];
        for (int i = 0; i < m + 2; i++) if (i != r) D[i][s] /= -D[r][s];
        D[r][s] = 1.0 / D[r][s];
        swap(B[r], N[s]);
    }

    bool Simplex(int phase) {
        int x = phase == 1 ? m + 1 : m;
        while (true) {
            int s = -1;
            for (int j = 0; j <= n; j++) {
                if (phase == 2 && N[j] == -1) continue;
                if (s == -1 || D[x][j] < D[x][s] || D[x][j] == D[x][s]
                    && N[j] < N[s]) s = j;
            }
            if (D[x][s] > -EPS) return true;
            int r = -1;
            for (int i = 0; i < m; i++) {
                if (D[i][s] < EPS) continue;
                if (r == -1 || D[i][n + 1] / D[i][s] < D[r][n + 1] /
                    D[r][s] ||
                    (D[i][n + 1] / D[i][s] == (D[r][n + 1] / D[r][s])
                     && B[i] < B[r])) r = i;
            }
            if (r == -1) return false;
            Pivot(r, s);
        }
    }

```

```

    DOUBLE Solve(VD &x) {
        int r = 0;
        for (int i = 1; i < m; i++) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -EPS) {
            Pivot(r, n);
            if (!Simplex(1) || D[m + 1][n + 1] < -EPS) return -
                numeric_limits<DOUBLE>::infinity();
            for (int i = 0; i < m; i++) if (B[i] == -1) {
                int s = -1;
                for (int j = 0; j <= n; j++)
                    if (s == -1 || D[i][j] < D[i][s] || D[i][j] == D[i][s]
                        && N[j] < N[s]) s = j;
                Pivot(i, s);
            }

```

```

            if (!Simplex(2)) return numeric_limits<DOUBLE>::infinity();
        }
        x = VD(n);
        for (int i = 0; i < m; i++) if (B[i] < n) x[B[i]] = D[i][n + 1];
        return D[m][n + 1];
    }

```

```

int main() {
    const int m = 4;
    const int n = 3;
    DOUBLE _A[m][n] = {
        { 6, -1, 0 },
        { -1, -5, 0 },
        { 1, 5, 1 },
        { -1, -5, -1 } };
    DOUBLE _b[m] = { 10, -4, 5, -5 };
    DOUBLE _c[n] = { 1, -1, 0 };
    VVD A(m);
    VD b(_b, _b + m);
    VD c(_c, _c + n);
    for (int i = 0; i < m; i++) A[i] = VD(_A[i], _A[i] + n);
    LPSolver solver(A, b, c);
    VD x;
    DOUBLE value = solver.Solve(x);
    cerr << "VALUE: " << value << endl; // VALUE: 1.29032
    cerr << "SOLUTION:"; // SOLUTION: 1.74194 0.451613 1
    for (size_t i = 0; i < x.size(); i++) cerr << " " << x[i]
        ;
}

```

33 Strongly Connected Component

```

int n;
vector<vector<int>>>g, gr;
vector<bool>used;
vector<int>order, component, scc;
vector<vector<int>>>comp;

```

```

void dfs1(int v){
    used[v]=1;
    for(auto u:g[v]){
        if(!used[u]){
            dfs1(u);
        }
    }
    order.pb(v);
}

```

```

void dfs2(int v){

```

```

used[v]=1;
component.pb(v);
for(auto u:gr[v]){
    if(!used[u]){
        dfs2(u);
    }
}
}

void scc_calc(){
    used.assign(n, false);
    for(int i=0; i<n; i++){
        if(!used[i]){
            dfs1(i);
        }
    }
    used.assign(n, false);
    scc.resize(n);
    for(int i=0, cnt=0; i<n; i++){
        int v=order[n-1-i];
        if(!used[v]){
            dfs2(v);
            for(auto u:component){
                scc[u]=cnt;
            }
            comp.pb(component);
            cnt++;
            component.clear();
        }
    }
}

```

34 Suffix Array

```

#define pb push_back
#define sz(x) (int) (x).size()
#define all(x) begin(x), end(x)
#define rep(i, a, b) for(int i=a; i<(b); ++i)
#define endl '\n'
typedef vector<int> vi;

void induced_sort(const vector<int> &vec, int val_range,
    vector<int> &SA, const vector<bool> &sl, const vector<
    int> &lms_idx) {
    vector<int> l(val_range, 0), r(val_range, 0);
    for (int c : vec) {
        if (c + 1 < val_range) ++l[c + 1];
        ++r[c];
    }
}

```

```

}
partial_sum(l.begin(), l.end(), l.begin());
partial_sum(r.begin(), r.end(), r.begin());
fill(SA.begin(), SA.end(), -1);
for (int i = lms_idx.size() - 1; i >= 0; --i)
    SA[--r[vec[lms_idx[i]]]] = lms_idx[i];
for (int i : SA)
    if (i >= 1 && sl[i - 1]) {
        SA[l[vec[i - 1]]++] = i - 1;
    }
}
fill(r.begin(), r.end(), 0);
for (int c : vec)
    ++r[c];
partial_sum(r.begin(), r.end(), r.begin());
for (int k = SA.size() - 1, i = SA[k]; k >= 1; --k, i =
    SA[k])
    if (i >= 1 && !sl[i - 1]) {
        SA[--r[vec[i - 1]]] = i - 1;
    }
}

vector<int> SA_IS(const vector<int> &vec, int val_range) {
    const int n = vec.size();
    vector<int> SA(n), lms_idx;
    vector<bool> sl(n);
    sl[n - 1] = false;
    for (int i = n - 2; i >= 0; --i) {
        sl[i] = (vec[i] > vec[i + 1] || (vec[i] == vec[i + 1]
            && sl[i + 1]));
        if (sl[i] && !sl[i + 1]) lms_idx.push_back(i + 1);
    }
    reverse(lms_idx.begin(), lms_idx.end());
    induced_sort(vec, val_range, SA, sl, lms_idx);
    vector<int> new_lms_idx(lms_idx.size(), lms_vec(lms_idx.
        size()));
    for (int i = 0, k = 0; i < n; ++i)
        if (!sl[SA[i]] && SA[i] >= 1 && sl[SA[i] - 1]) {
            new_lms_idx[k++] = SA[i];
        }
    int cur = 0;
    SA[n - 1] = cur;
    for (size_t k = 1; k < new_lms_idx.size(); ++k) {
        int i = new_lms_idx[k - 1], j = new_lms_idx[k];
        if (vec[i] != vec[j]) {
            SA[j] = ++cur;
            continue;
        }
        bool flag = false;
        for (int a = i + 1, b = j + 1;; ++a, ++b) {
            if (vec[a] != vec[b]) {

```

```

                flag = true;
                break;
            }
            if ((!sl[a] && sl[a - 1]) || (!sl[b] && sl[b -
                1])) {
                flag = !((!sl[a] && sl[a - 1]) && (!sl[b] &&
                    sl[b - 1]));
                break;
            }
        }
        SA[j] = (flag ? ++cur : cur);
    }
    for (size_t i = 0; i < lms_idx.size(); ++i)
        lms_vec[i] = SA[lms_idx[i]];
    if (cur + 1 < (int)lms_idx.size()) {
        auto lms_SA = SA_IS(lms_vec, cur + 1);
        for (size_t i = 0; i < lms_idx.size(); ++i) {
            new_lms_idx[i] = lms_idx[lms_SA[i]];
        }
    }
    induced_sort(vec, val_range, SA, sl, new_lms_idx);
    return SA;
}

vector<int> suffix_array(const string &s, const int LIM =
    128) {
    vector<int> vec(s.size() + 1);
    copy(begin(s), end(s), begin(vec));
    vec.back() = '$';
    auto ret = SA_IS(vec, LIM);
    ret.erase(ret.begin());
    return ret;
}

vector<int> LCP(const string &s, const vector<int> &sa) {
    int n = s.size(), k = 0;
    vector<int> lcp(n), rank(n);
    for (int i = 0; i < n; ++i)
        rank[sa[i]] = i;
    for (int i = 0; i < n; ++i, k ? k-- : 0) {
        if (rank[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = sa[rank[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k
            ])
            k++;
        lcp[rank[i]] = k;
    }
    lcp[n - 1] = 0;
}

```

```

    return lcp;
}

```

35 Template

```

#pragma GCC optimize ("-O2")
#include <bits/stdc++.h>
using namespace std;
#define fastio ios_base::sync_with_stdio(0);cin.tie(0)
#define pb push_back
#define mp make_pair
#define fi first
#define se second
#define all(x) x.begin(),x.end()
#define memreset(a) memset(a,0,sizeof(a))
#define testcase(t) int t;cin>>t;while(t-->0)
#define forstl(i,v) for(auto &i: v)
#define forn(i,e) for(int i=0;i<e;++i)
#define forsn(i,s,e) for(int i=s;i<e;++i)
#define rforn(i,s) for(int i=s;i>=0;--i)
#define rforsn(i,s,e) for(int i=s;i>=e;--i)
#define bitcount(a) __builtin_popcount(a) // set bits (add ll)
#define ln '\n'
#define getcurrtime() cerr<<"Time = "<<((double)clock()/CLOCKS_PER_SEC)<<endl
#define dbgarr(v,s,e) cerr<<#v<<" = "; forsn(i,s,e) cerr<<v[i]<<" "; cerr<<endl
#define dbg(args...) { string _s = #args; replace(_s.begin(), _s.end(), ',', ' '); stringstream _ss(_s); istream_iterator<string> _it(_ss); err(_it, args); }
void err(istream_iterator<string> it) { cerr<<endl; }
template<typename T, typename... Args>
void err(istream_iterator<string> it, T a, Args... args) {
    cerr << *it << " = " << a << "\t"; err(++it, args...);
}
template<typename T1,typename T2>
ostream& operator <<(ostream& c,pair<T1,T2> &v){
    c<<"("<<v.fi<<" "<<v.se<<">"; return c;
}
template <template <class...> class TT, class ...T>
ostream& operator<<(ostream& out,TT<T...>& c){
    out<<"{ ";
    forstl(x,c) out<<x<<" ";
    out<<"}"; return out;
}
typedef long long ll;

```

```

typedef unsigned long long ull;
typedef long double ld;
typedef pair<ll,ll> p64;
typedef pair<int,int> p32;
typedef pair<int,p32> p96;
typedef vector<ll> v64;
typedef vector<int> v32;
typedef vector<v32> vv32;
typedef vector<v64> vv64;
typedef vector<p32> vp32;
typedef vector<p64> vp64;
typedef vector<vp32> vvp32;
typedef map<int,int> m32;
const int LIM=2e5+5,MOD=1e9+7;
const ld EPS = 1e-9;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

int main(){
    fastio;

    return 0;
}

```

36 Treap

```

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// mt19937 rng(0);

struct item{
    int prior, cnt;
    ll value;
    ll sum_val;
    int num;
    bool rev;
    item *l, *r, *par;
    item(int value, int prior): value(value), prior(prior), rev(0), cnt(1), l(NULL), r(NULL), par(NULL), sum_val(value) {}
    // prior = priority
    // value = value of the node
    // cnt = subtree size
    //
}

```

```

};
typedef struct item* pitem;

int cnt(pitem it){
    // Find subtree size
    return it ? it->cnt : 0;
}

ll val(pitem it){
    return it ? it->sum_val : 0;
}

void upd_cnt(pitem it){
    if(it){
        // recomputing after operations
        it->cnt = cnt(it->l) + cnt(it->r) + 1;
        it->sum_val = val(it->l) + val(it->r) + it->value;
        if(it->l){
            it->l->par = it;
        }
        if(it->r){
            it->r->par = it;
        }
        it->par = NULL;
    }
}

void heapify(pitem t){
    if(!t) return;
    pitem max = t;
    if(t->l != NULL && t->l->prior > max->prior){
        max = t->l;
    }
    if(t->r != NULL && t->r->prior > max->prior){
        max = t->r;
    }
    if(max != t){
        swap(t->prior, max->prior);
        heapify(max);
    }
}

pitem build(int *a, int n){
    if(n == 0) return NULL;
    int mid = n / 2;
    pitem t = new item(a[mid], rng());
    t->l = build(a, mid);
    t->r = build(a+mid+1, n-mid-1);
    heapify(t);
    upd_cnt(t);
}

```

```

    return t;
}

void push(pitem it){
    if(it && it->rev){
        it->rev = false;
        swap(it->l, it->r);
        if(it->l) it->l->rev ^= true;
        if(it->r) it->r->rev ^= true;
    }
}

void merge(pitem &t, pitem l, pitem r){
    push(l);
    push(r);
    if(!l || !r){
        t = l ? l : r;
    }
    else if(l->prior > r->prior){
        merge(l->r, l->r, r), t = l;
    }
    else{
        merge(r->l, l, r->l), t = r;
    }
    upd_cnt(t);
}

void split(pitem t, pitem &l, pitem &r, int key, int add = 0){
    if(!t){
        return void(l=r=0);
    }
    push(t);
    int cur_key = add + cnt(t->l);
    if(key <= cur_key){
        split(t->l, l, t->l, key, add), r = t;
    }
    else{
        split(t->r, t->r, r, key, add+1+cnt(t->l)), l = t;
    }
    upd_cnt(t);
    // upd_cnt(l);
    // upd_cnt(r);
}

void reverse(pitem t, int l, int r){
    pitem t1, t2, t3;
    split(t, t1, t2, l);
    split(t2, t2, t3, r-1+1);
    t2-> rev ^= true;
}

```

```

    merge(t, t1, t2);
    merge(t, t, t3);
}

v32 v;

void traversal(pitem t){
    if(!t) return;
    push(t);
    traversal(t->l);
    v.pb(t->value);
    traversal(t->r);
}

pitem get_par(pitem t){
    pitem n = t->par;
    upd_cnt(t);
    if(n == NULL){
        return t;
    }
    return get_par(n);
}

void inord_traversal(pitem t){
    v.clear();
    traversal(t);
}

// in main
int q; cin>>q;

pitem t[q];
int a[1] = {0};
for(n,i,q){
    t[i] = build(a, 1);
}

```

37 Two SAT

```

// (a1 v a2) is split into (not a1 implies a2) and (not a2
// implies a1)
int n;//2*N
vector<vector<int>>>adj;
vector<vector<int>>>g, gt;//g stores implication, gt stores
reverse edges
vector<bool>used;
vector<int>order, comp;

```

```

vector<bool>assignment;//resize all to n, //assignment is
for i=1 to (n/2)

void dfs1(int v){
    used[v]=1;
    for(int u:g[v]){
        if(!used[u]){
            dfs1(u);
        }
    }
    order.pb(v);
}

void dfs2(int v, int c1){
    comp[v]=c1;
    for(int u:gt[v]){
        if(comp[u]==-1){
            dfs2(u, c1);
        }
    }
}

bool solve(){
    used.assign(n, false);
    for(int i=0; i<n; i++){
        if(!used[i]){
            dfs1(i);
        }
    }

    comp.assign(n, -1);
    for(int i=0, j=0; i<n; ++i){
        int v=order[n-i-1];
        if(comp[v]==-1){
            dfs2(v, j++);
        }
    }

    assignment.assign(n/2, false);
    for(int i=0; i<n; i=i+2){
        if(comp[i]==comp[i+1]){
            return false;
        }
        assignment[i/2]=comp[i]>comp[i+1];
    }
    return true;
}

```

38 Z and KMP

```
vector<int> z_function(string s) {
    int n = (int) s.length();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
```

39 Z ideas

Gray codes Applications:

1. Gray code of n bits forms a Hamiltonian cycle on a hypercube, where each bit corresponds to one dimension.
2. Gray code can be used to solve the Towers of Hanoi problem. Let n denote number of disks. Start with Gray code of length n which consists of all zeroes ($G(0)$) and move between consecutive Gray codes (from $G(i)$ to $G(i+1)$).

Let i -th bit of current Gray code represent n -th disk (the least significant bit corresponds to the smallest disk and the most significant bit to the biggest disk). Since exactly one bit changes on each step, we can treat changing i -th bit as moving i -th disk. Notice that there is exactly one move option for each disk (except the smallest one) on each step (except start and finish

positions).

There are always two move options for the smallest disk but there is a strategy which will always lead to answer: if n is odd then sequence of the smallest disk moves looks like ftrftr ... where f is the initial rod, t is the terminal rod and r is the remaining rod), and if n is even: frtfrt

```
int gray (int n) {return n ^ (n >> 1);}
int rev_g (int g) {
    int n = 0;
    for (; g; g >>= 1) n ^= g;
    return n;}
```

Enumerating all submasks of a bitmask:

```
for (int s=m; ; s=(s-1)&m) {
    ... you can use s ...
    if (s==0) break;}
```

Sparse Table:

```
int st[MAXN][K];
for (int i = 0; i < N; i++) st[i][0] = array[i];
for (int j = 1; j <= K; j++)
    for (int i = 0; i + (1 << j) <= N; i++)
        st[i][j] = min(st[i][j-1], st[i + (1 << (j - 1))][j - 1]);
```

To get minimum of a range:

```
int j = log[R - L + 1];
int minimum = min(st[L][j], st[R - (1 << j) + 1][j]);
```

Divide and Conquer DP:

Some dynamic programming problems have a recurrence of this form:
 $dp(i, j) = \min_k \{dp(i, k) + C(k, j)\}$ where $C(k, j)$ is some cost function.

Say $1 \leq i \leq n$ and $1 \leq j \leq m$, and evaluating C takes $O(1)$ time. Straightforward evaluation of the above recurrence is $O(nm^2)$.

There are nm states, and m transitions for each state.

Let $opt(i, j)$ be the value of k that minimizes the above expression.

If $opt(i, j)$ $opt(i, j+1)$ for all i, j , then we can apply divide-and-conquer DP. This known as the monotonicity condition.

The optimal "splitting point" for a fixed i increases as j increases.

```
int n;
long long C(int i, int j);
```

```
vector<long long> dp_before(n), dp_cur(n);
// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int optr){
    if (l > r)
        return;
    int mid = (l + r) >> 1;
    pair<long long, int> best = {INF, -1};
    for (int k = optl; k <= min(mid, optr); k++) {
        best = min(best, {dp_before[k] + C(k, mid), k});
    }
    dp_cur[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);}
```

Knuth Optimization:

```
dp[i][j] = mini < k < j {dp[i][k] + dp[k][j]} + C[i][j]
monotonicity : C[b][c] <= C[a][d]
quadrangle inequality: C[a][c]+C[b][d] <= C[a][d]+C[b][c]
```

Lyndon factorization: We can get the minimum cyclic shift. Factorize the string as $s = w_1 w_2 w_3 \dots w_n$

```
string min_cyclic_string(string s) {
    s += s;
    int n = s.size();
    int i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k)
            i += j - k;
    }
    return s.substr(ans, n / 2);}
```

Generating all k -subsets:

```
vector<int> ans;
void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0) return;
    if (!n) {
        for (int i = 0; i < idx; ++i) {
            if (ans[i]) cout << i + 1;
        }
    }
}
```

```

    cout << "\n";
    return;}
ans[idx] = rev;
gen(n - 1, k - rev, idx + 1, false);
ans[idx] = !rev;
gen(n - 1, k - !rev, idx + 1, true);}
void all_combinations(int n, int k) {
    ans.resize(n);gen(n, k, 0, false);}

```

Simpsons formula for integration:

```

const int N = 1000 * 1000; // number of steps (already
    multiplied by 2)
double simpson_integration(double a, double b){
    double h = (b - a) / N;
    double s = f(a) + f(b); // a = x_0 and b = x_2n
    for (int i = 1; i <= N - 1; ++i) { // Refer to final
        Simpson's formula
        double x = a + h * i;
        s += f(x) * ((i & 1) ? 4 : 2);}
    s *= h / 3;
    return s;}

```

Picks theorem:

Given a certain lattice polygon with non-zero area. We denote its area by S , the number of points with integer coordinates lying strictly inside the polygon by I and the number of points lying on polygon sides by B . Then, the Pick formula states: $S = I + B/2 - 1$. In particular, if the values of I and B for a polygon are given, the area can be calculated in $O(1)$ without even knowing the vertices.

FFT Matrices:

XOR FFT: $1 \ 1 \ / \ 1 \ -1$, AND FFT: $0 \ 1 / \ 1 \ 1$, OR FFT: $1 \ 1 / \ 1 \ 0$

Harmonic lemma:

```

for (int i = 1, la; i <= n; i = la + 1) {
    la = n / (n / i);
    v.pb(mp(n/i, la-i+1));}
//n / x yields the same value for i <= x <= la.

```

Mobius inversion theory:

if f and g are multiplicative, then their dirichlet convolution, i.e $\sum_{d|x} f(d)g(x/d)$ is also multiplicative. eg. choose $g = 1$

Properties:

1. If $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} g(d)u(n/d)$.
2. $\sum_{d|n} u(d) = [n=1]$

Standard question: Number of co-prime integers in range $1, n$
 Answer: $f(n) = \sum_{d=1}^n u(d) \text{floor}(n/d)^2$

Euler totient: $\phi(\text{totient fn}) = u * n$ (dirichlet convolution)

a Nim position $(n_1, \dots, n_k)$ is a second player win in misere Nim if and only if some $n_i > 1$ and $n_1 \text{ xor } \dots \text{ xor } n_k = 0$, or all $n_i \leq 1$ and $n_1 \text{ xor } \dots \text{ xor } n_k = 1$.

Fibonacci Identities:

1. $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$
2. $F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$
3. $F_n \mid F_m \Leftrightarrow n \mid m$
4. $\text{GCD}(F_m, F_n) = F_{\text{gcd}(m, n)}$
5. $F_{2k} = F_k(2F_{k+1} - F_k)$. $F_{2k+1} = F_{k+1}^2 + F_k^2$
6. $n \geq \phi(m) \Rightarrow x^n = x^{(\phi(m) + n \% \phi(m)) \bmod m}$

Counting labeled graphs:

The total number of labelled graphs is $G_n = 2^{\binom{n-1}{2}}$
 Number of connected labelled graphs is $C_n = G_n - 1/n * (\sum_{k=1}^{n-1} k \cdot \binom{n}{k} \cdot C_k \cdot G_{n-k})$
 Number of labelled graphs with k components: $D[n][k] = \sum_{s=1}^n ((n-1)C(s-1)) C_s D[n-s][k-1]$

Steiner tree dp:

The idea is to build a dynamic programming $DP[i][m]$, where i is which vertex you are at and m is a bitmask of which capitals you joined. You can preprocess the APSP (Floyd-Warshall, or many Dijkstras because of the small constants) and calculate $DP[i][m]$ like this: $DP[i][m] =$

$\min(DP[i][s] + DP[j][m-s] + \text{dist}[i][j])$, with s being a submask of m . In the end the complexity is $O(3^k * n^2)$. To get $O(3^k * n)$ complexity, you do 2 transitions: 1. $O(3^k * n)$ transition using submasks and 2. $O(2^k * n^2)$ transition, that is, $O(n^2)$ transition for each mask.

Sum of subsets DP:

$F(x) = \text{sum of all } A(i) \text{ such that } x \& i = i$.

```

//iterative version
for(int mask = 0; mask < (1<<N); ++mask){
    dp[mask][1] = A[mask]; //handle base case separately (leaf
        states)
    for(int i = 0; i < N; ++i){
        if(mask & (1<<i))
            dp[mask][i] = dp[mask][i-1] + dp[mask^(1<<i)][i-1];
        else
            dp[mask][i] = dp[mask][i-1];}
    F[mask] = dp[mask][N-1];}

```

//memory optimized, super easy to code.

```

for(int i = 0; i < (1<<N); ++i) F[i] = A[i];
for(int i = 0; i < N; ++i) for(int mask = 0; mask < (1<<N);
    ++mask){
    if(mask & (1<<i)) F[mask] += F[mask^(1<<i)];}

```

15 puzzle problem: existence of solution

```

int a[16];
for (int i=0; i<16; ++i)
    cin >> a[i];
int inv = 0;
for (int i=0; i<16; ++i)
    if (a[i])
        for (int j=0; j<i; ++j)
            if (a[j] > a[i])
                ++inv;
for (int i=0; i<16; ++i)
    if (a[i] == 0)
        inv += 1 + i / 4;
puts ((inv & 1) ? "No Solution" : "Solution Exists");

```